AID 304 Big Data Analytics

Hadoop Architecture

Week 3, Lecture 3

Dr. Kazeem Oladejo

March 2026

Course Assessment - Before

- Graded Lab 1 (5 %)
- In-Term Quiz (10%)
- Mid-Term Exam (30 %)
- Graded Lab 2 (5 %)
- Project (10 %)
- Final Exam (40 %)

Course Assessment - After

- Graded Lab 1 (10 %)
- In-Term Quiz (10 %)
- Mid-Term Exam (25 %)
- Graded Lab 2 (10 %)
- Project (10 %)
- Final Exam (35 %)

What is Apache Hadoop?

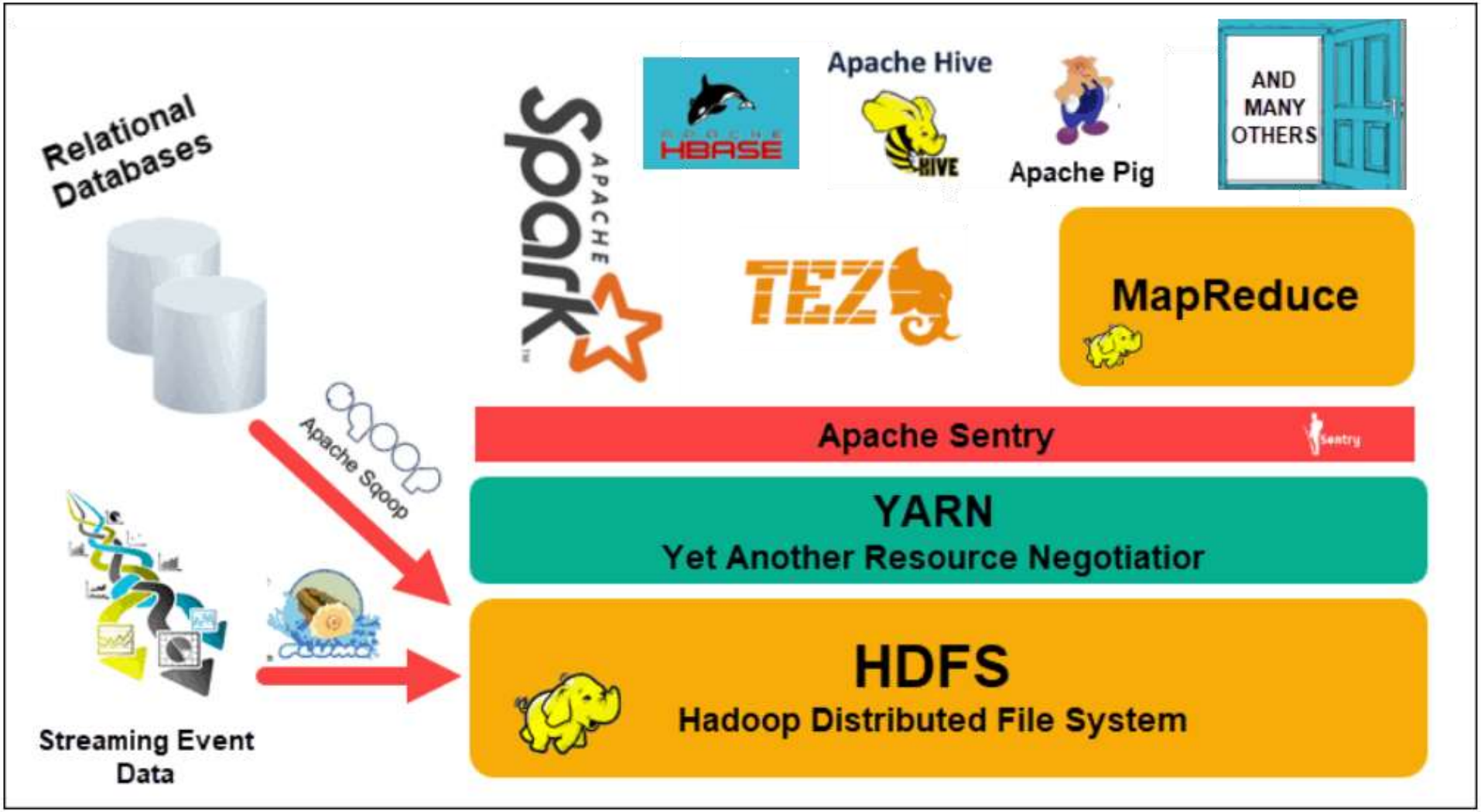
Apache Hadoop is an open-source framework and platform designed for distributed storage and processing of massive, complex datasets across clusters of commodity hardware. Hadoop enables fast, parallel computing using ***programming and storage models that provide fault tolerance and horizontal scalability***.

History of Apache Hadoop

Hadoop was created by Doug Cutting and Mike Cafarella in 2005 based on a Google File System paper that was published in October 2003. It was originally developed to support Apache Nutch (an open-source web crawler product) and later moved to a separate sub-project. Doug, who was working at Yahoo! at the time named the project after his son’s toy elephant “Hadoop”. In 2008, Hadoop was open sourced, and The Apache Software Foundation made Hadoop available to the public in

Ref: https://community.hasc.com/communities/big-data-analytics/brief-over

The Hadoop Architecture



Ref: https://phoenixnap.com/kb/apache-hadoop-architecture-explaine

The Hadoop Platform

- Provides a low-cost Big Data platform, that is open source and uses cloud services.
- Hadoop enables distributed processing of large datasets across clusters of computers using a programming models. Terabytes of data processing can take just a few minutes.
- Linux and Java based
- Core components are Hadoop Distributed File System (HDFS), Yet Another Resource Negotiator (YARN) and MapReduce.

The Hadoop Platform Characteristics

- Scalable: can be scaled up (enhanced) by adding storage and processing units to regular commodity cost as requirements change.
- Self-manageable: creation of storage and processing resources which are used, scheduled and reduced or increased with the help of the system itself
- Self-healing: detect and handle failures at the task level and communicate such failures and resolutions.
- Distributed Computing

Distributed Computing Model

- Hadoop can run multiple application tasks in a distributed mode using several computers associated with multiple data nodes, executing the tasks at the same time in parallel.
- Computing model consists of two components:
  - data stored in blocks in the clusters (set of computers)
  - computations at each individual cluster in parallel with another.

Cluster Infrastructure - with Server Racks

Ref: https://www.linkedin.com/pulse/infrastructure-hadoop-cluster-part-2-saheel-shah

Hadoop Distributed File System

Hadoop Distributed File System (HDFS) is the core of Hadoop. It is an open version of Google's File System. HDFS is a distributed, scalable file system with built-in redundancy. The distributed file system is built on top of many nodes, each containing a regular file system.

HDFS was designed to scale and hold petabytes of data. As such, a few assumptions were designed for the system:

- sequential reads: fast with full data scanning support
- communicates the location of the data so that computation can be moved to the data rather than the other way around
- node failure should be tolerated in software.

Hadoop Storage Model

- Each node in a Hadoop cluster has its own disk space, memory, bandwidth, and processing.
- Incoming data is split into individual data blocks, which are then stored within the HDFS distributed storage layer.
- HDFS stores three copies of each data set throughout the cluster as replicas.
- The HDFS master node (aka NameNode) keeps the metadata for the individual data block and all its replicas.
- The default data block size in Hadoop 3 (HDFS) is 128 MB. This size is designed to minimize disk seek times and reduce the metadata overhead managed by the NameNode.

Fault Tolerance

- Hadoop is fault tolerant due to replication
- Data partitioned into data blocks and written onto a set of data nodes.
- If a fault occurs, Hadoop uses the replicated node to make the data available.
- Follows CAP theorem— out of three properties (consistency, availability and partitions), two must at least be present

DataNode

DataNodes are the slave nodes that store the actual data blocks of files and perform read, write, and replication operations as instructed by the master NameNode.

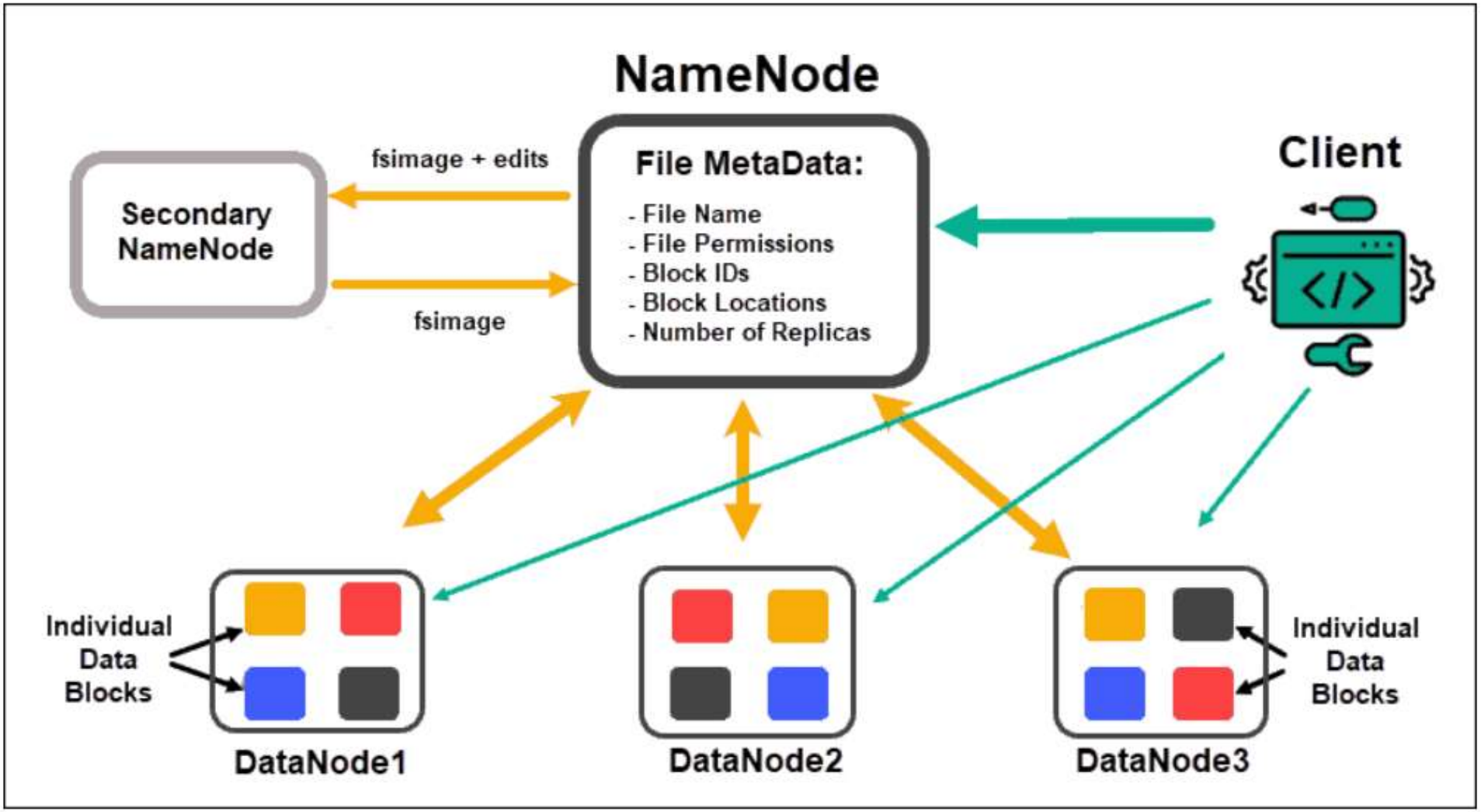
- Data Storage: stores HDFS data as blocks in their local file systems. Each block is a separate file on the local disk.
- Serving Client Requests: handle read and write requests directly from HDFS clients, guided by metadata provided by the NameNode.
- Block Management and Replication: Upon instruction from the NameNode, DataNodes create, delete, and replicate data blocks to maintain the configured replication factor (default is three) and ensure data redundancy.

NameNode

Hadoop NameNode is the master node in HDFS that manages the file system namespace and acts as the central metadata repository.

- The file metadata for these blocks, which include the file name, file permissions, IDs, locations, and the number of replicas, are stored in a fsimage
- Standby NameNode is an automated

NameNode


NameNode
Secondary NameNode
fsimage + edits
fsimage
File MetaData:
- File Name
- File Permissions
- Block IDs
- Block Locations
- Number of Replicas
Client
Individual Data Blocks
DataNode1
DataNode2
DataNode3
Individual Data Blocks

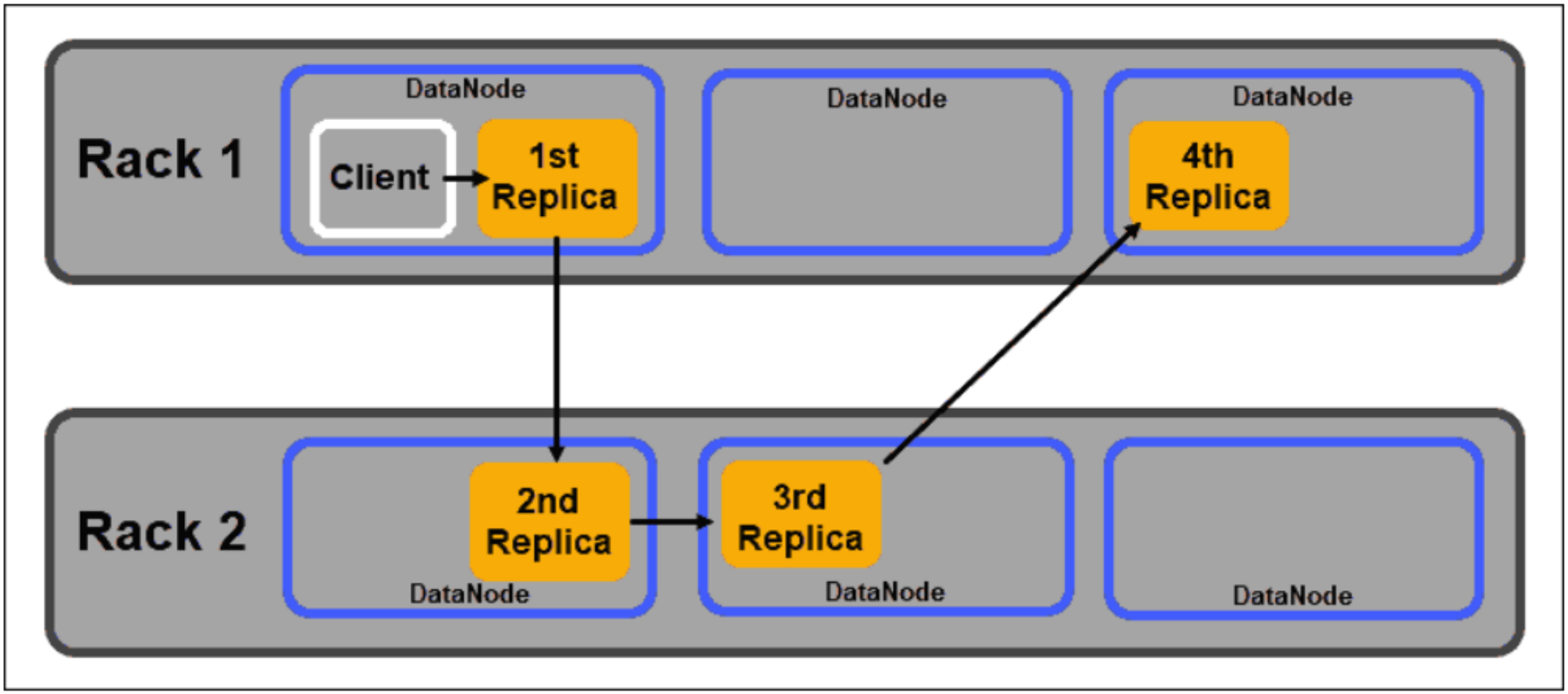
Rack Aware Placement

The HDFS NameNode maintains a default rack-aware replica placement policy:

- The first data block replica is placed on the same node as the client.
- The second replica is automatically placed on a random DataNode on a different rack.
- The third replica is placed in a separate DataNode on the same rack as the second replica.

Rack placement policy may maintain only one replica per node and sets a limit of two replicas per server

Rack Aware Placement



YARN (Yet Another Resource Negotiator)

YARN manages the allocation of compute resources and schedules user applications an efficient manner for the Hadoop platform.

YARN Resource management entails scheduling tasks such that data locality is maximized and that resources are not starved by large jobs.

YARN Architecture

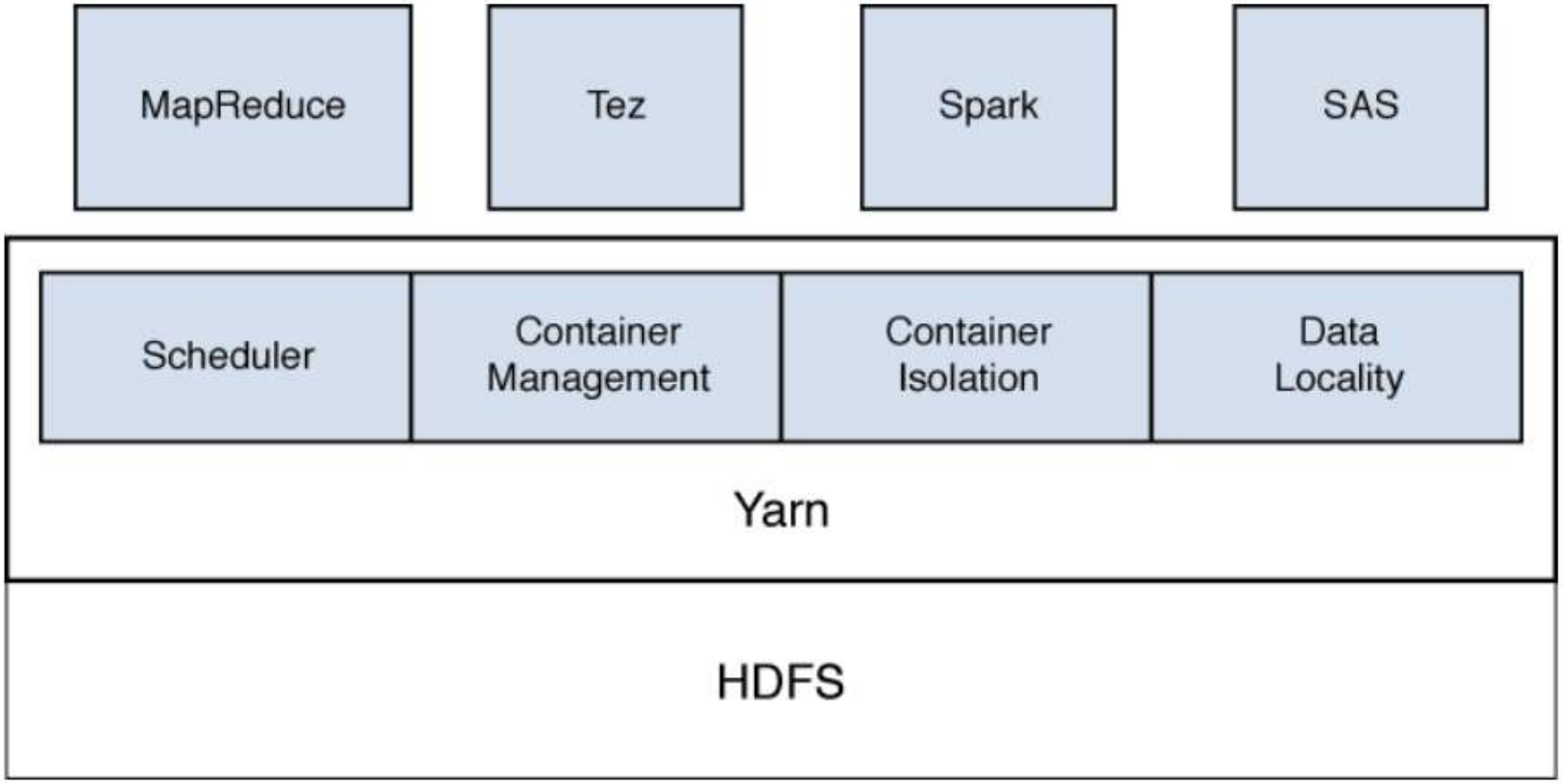


Figure 3.2 Hadoop architecture with HDFS, Yarn, and the various processing engines—MapReduce, Spark, Tez.

Containers

- Processing resources in a Hadoop cluster are always deployed in containers. A container has memory, system files, and processing space.
- YARN partitions resources into containers that at the base level are a CPU core and an amount of memory. YARN monitors running containers to ensure that the memory, CPU, disk, and network quotas requested by the task are not exceeded
- Container processes on a slave node are initially provisioned, monitored, and tracked by the NodeManager on that specific slave node.

Data Locality

YARN provides data locality as a resource. That is, a YARN job (e.g., MapReduce) can request that specific containers run on servers that host specific data (or as close to where data reside in the cluster as possible).

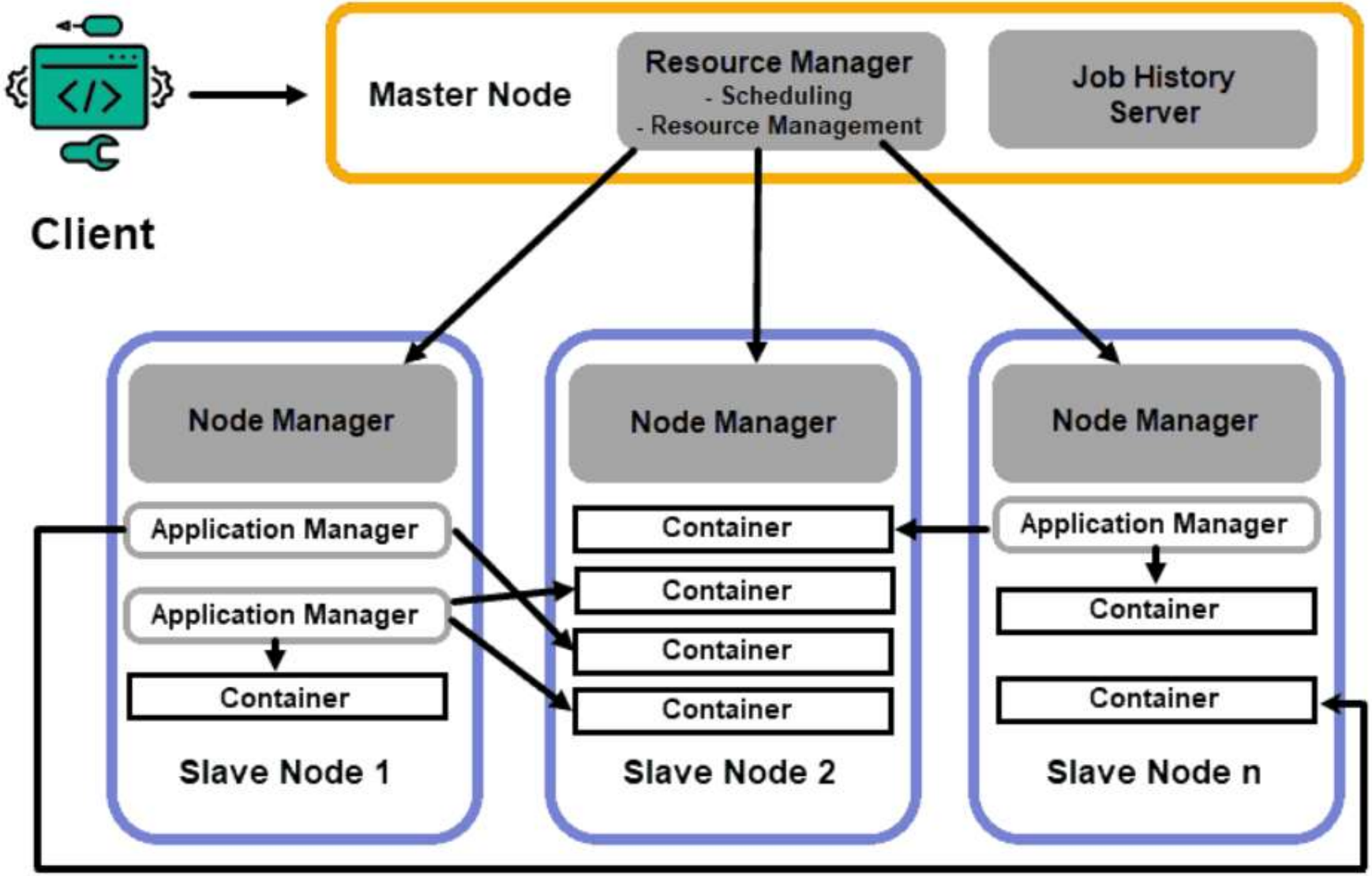
Container Isolation

Data locality provides a level of control that is important to ensure that a distributed system continues to run smoothly, resources are shared in a fair fashion, containers can be private (***isolated from other users***), enabling users to have their tasks scheduled in a timely manner.

Scheduler (ResourceManager)

The ResourceManager (RM)’s sole focus is on scheduling workloads. The RM daemon controls all the processing resources in a Hadoop cluster. Its primary purpose is to designate resources to individual applications located on the slave nodes. It maintains a global overview of the ongoing and planned processes, handles resource requests, and schedules and assigns resources accordingly.

YARN Workflow



NodeManager

Each slave node has a NodeManager processing service and a DataNode storage service. Together they form the backbone of a Hadoop distributed system.

The DataNode, as mentioned previously, is an element of HDFS and is controlled by the NameNode. The NodeManager, in a similar fashion, acts as a slave to the ResourceManager. The primary function of the NodeManager daemon is to track processing-resources data on its slave node and send regular reports to the ResourceManager.

The Application Master

Every container on a slave node has its dedicated Application Master. Application Masters are deployed in a container as well. For example, MapReduce has an Application Master that executes map and reduce tasks.

An Application Master sends messages to the Resource Manager about its current status and the state of the application it monitors. Based on the provided information, the Resource Manager schedules additional resources or assigns them elsewhere in the cluster if they are no longer needed.

The Application Master oversees the full lifecycle of an application, from requesting the needed containers from the RM to submitting container lease requests to the NodeManager.

The JobHistory Server

The JobHistory Server allows users to retrieve information about applications that have completed their activity.

JobHistory uses the REST API to provide interoperability and can dynamically inform users on current and completed jobs served by the server in question.

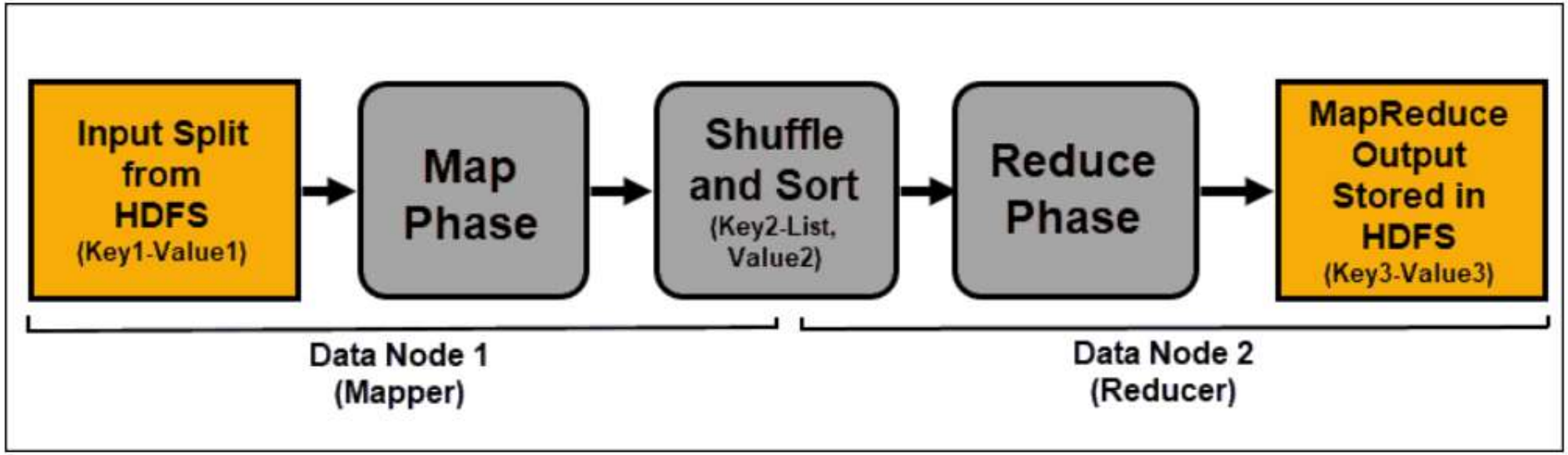
MapReduce

MapReduce is a programming framework that processes data stored across the Hadoop cluster. MapReduce is defined as the distributed processing model whereby a problem can be broken down into three phases: a map phase, a shuffle phase, and a reduce phase.

1. **Map phase:** input data are processed in parallel across the cluster through a method that transforms raw data into keys and values.
2. **Shuffle phase:** The keys are then sorted and “shuffled” into buckets by common, like keys (i.e., all values with the same key are guaranteed to go to the same reducer).
3. **Reduce phase:** The reducers then process the values for every key, often storing the results either back on HDFS or some other persistent store.

NOTE: MapReduce relies on the data locality characteristics of HDFS and the task/resource management of YARN to efficiently run this three-step computation.

MapReduce Workflow



The Hadoop servers that perform the mapping and reducing tasks are often referred to as **Mappers** and **Reducers**.

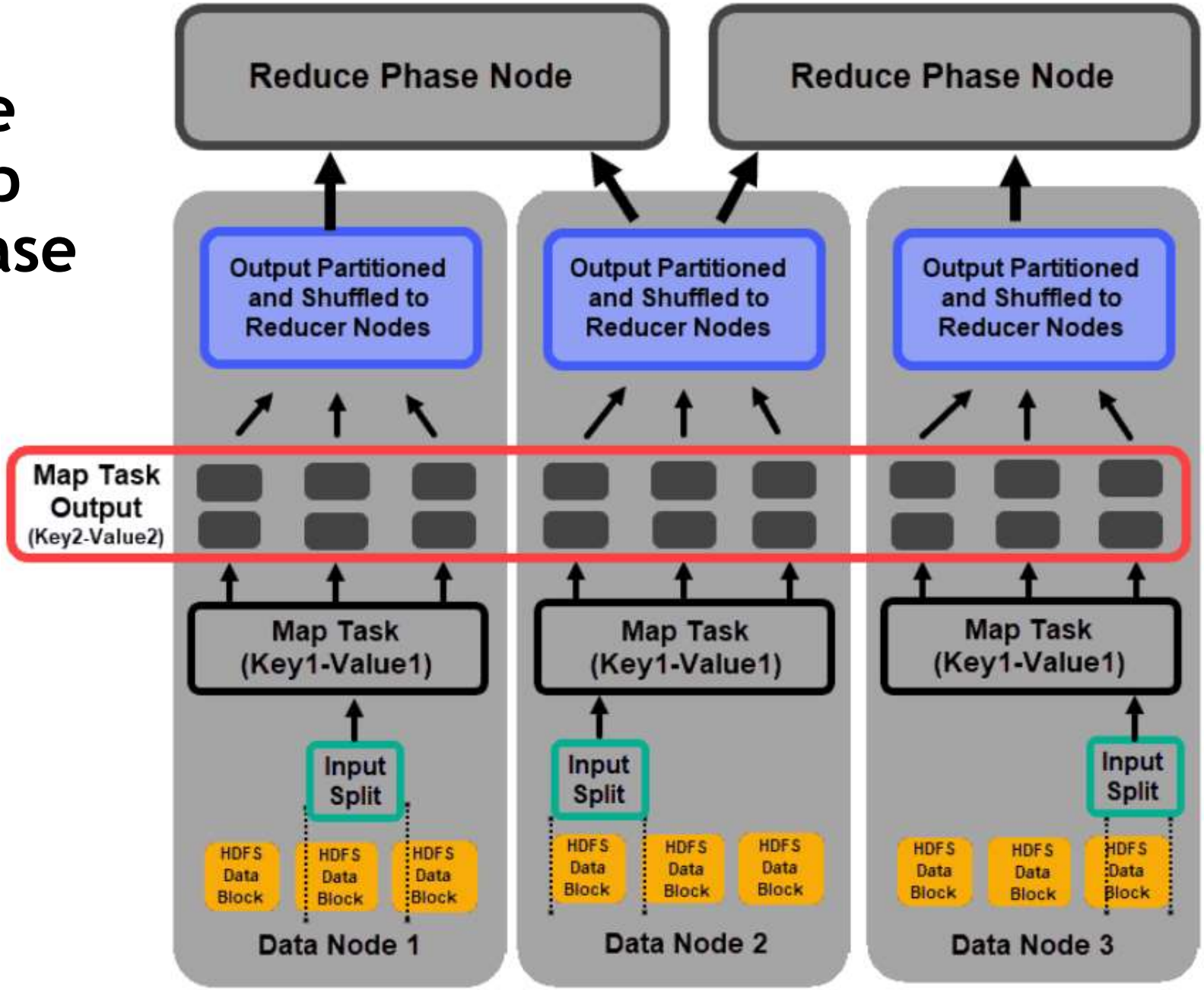
The ResourceManager decides how many mappers to use. This decision depends on the size of the processed data and the memory block available on each mapper server.

The Map Phase

The mapping process ingests individual logical expressions of the data stored in the HDFS data blocks. These expressions can span several data blocks and are called ***input splits***. Input splits are introduced into the mapping process as key-value pairs.

A mapper task goes through every key-value pair and creates a new set of key-value pairs, distinct from the original input data. The complete assortment of all the key-value pairs represents the output of the mapper task.

The Map Phase



The Shuffle (and Sort) Phase

Shuffle is a process in which the results from all the map tasks are copied to the reducer nodes. The copying of the map task output is the only exchange of data between nodes during the entire MapReduce job.

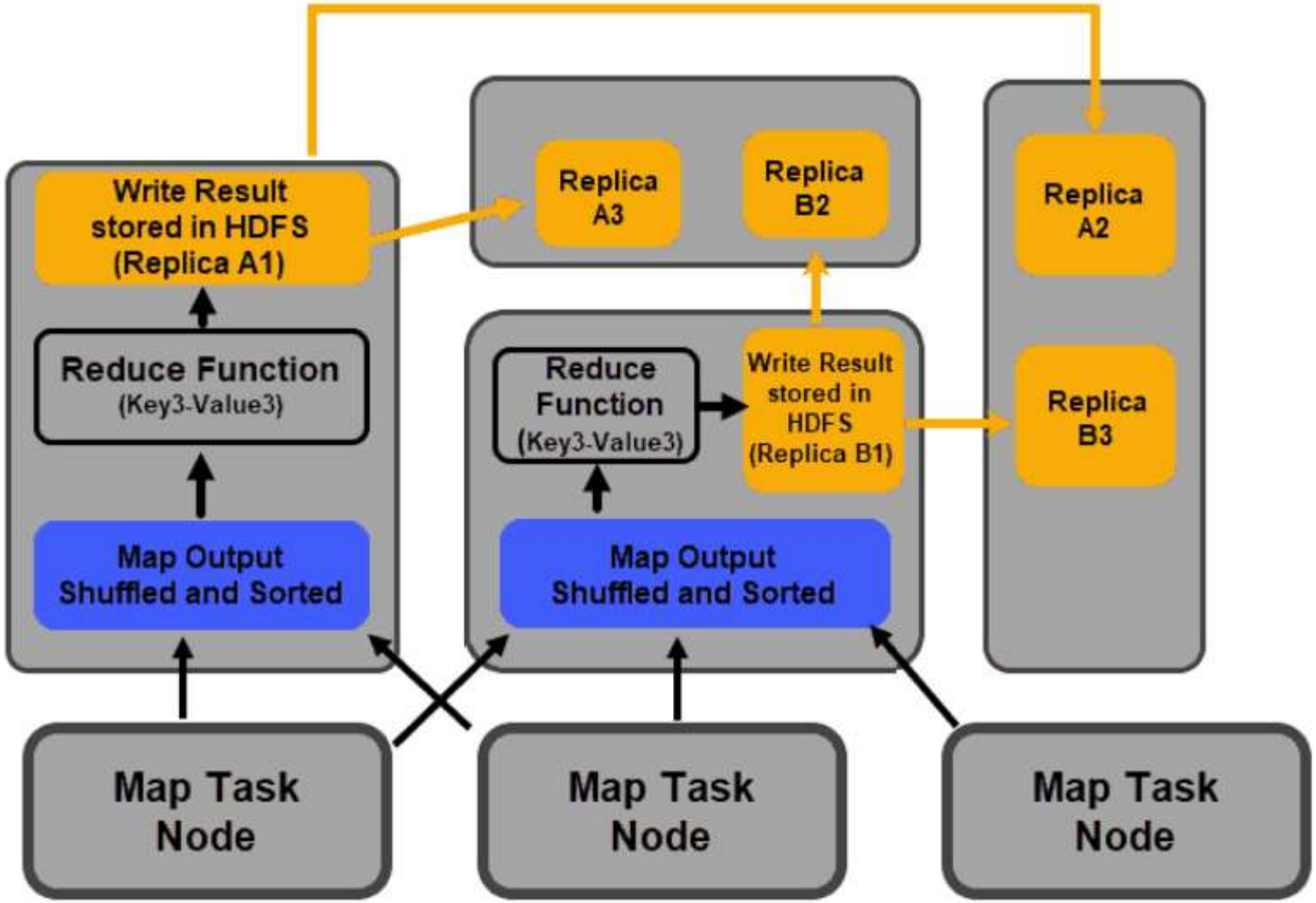
The output of a map task needs to be arranged to improve the efficiency of the reduce phase. The mapped key-value pairs, being shuffled from the mapper nodes, are arrayed by key with corresponding values. A reduce phase starts after the input is sorted by key in a single input file.

The shuffle and sort phases run in parallel. Even as the map outputs are retrieved from the mapper nodes, they are grouped and sorted on the reducer nodes.

The Reduce Phrase

The map outputs, already shuffled and sorted into a single reduce input file are used by a reduce function to aggregate the values based on the corresponding mapped keys. The output from the reduce process is a new key-value pair. This result represents the output of the entire MapReduce job and is, by default, stored in HDFS.

The Reduce Phrase



A MapReduce Example

The canonical example of MapReduce is counting word occurrences in a large corpus of text. To see how this fits, the case study of millions of “accident reports” passed through each phase of the MapReduce process for this problem.

1) **Mapper**: given that the text data were loaded into HDFS, it was automatically “sliced,” replicated, and distributed across the HDFS servers. Each slice is scanned in parallel to count words using a key/value pair (i.e., each occurrence of a word creates a key value pair of the form (car,10) where “car” has been found 10 times in that slice). This key/value mapping gets generated for each occurrence of “car”.

A MapReduce Example

2) **Shuffle**: When the counting is complete, all identical keys are shuffled from their mapper processes to a reducer process.

3) **Reducer**: The inputs to the reducer process are all the key/value pairs for a specific word. In the case of word counting, the reducer just sums the values associated with the word. Finally, the reducer will write out the word and the total count associated with it for inspection, for example (car, 102,314).

Hadoop Tez

Apache Tez was developed to address the fact that many problems extend beyond one MapReduce phase into multiple reduce steps and sometimes do not need data sorted between these steps. Tez enables a more efficient implementation of complex data flows and more efficient joining of sets of data within the native Hadoop SQL engine, Hive. Key notes:

- Tez is a lower-level software API for use by other projects (not an end-user tool).
- The Tez model enables, data to be transferred directly from the reducer of one job to another reducer without writing out intermediate data to HDFS (in memory).
- Tez enable computationally expensive multi-way joins to be better represented as a directed acyclic graph (DAG) of reducers rather than a linear pipeline of mappers and reducers.

Hadoop User Permissions

The Kerberos network protocol is the chief authorization system in Hadoop. It makes sure that only verified nodes and users have access and operate within the cluster.

However, it is a good idea to use additional security frameworks such as **Apache Sentry** or **Ranger**. These tools help you manage all security-related tasks from a central, user-friendly environment.

Scaling Hadoop (Hardware)

The **NameNode** is a vital element of your Hadoop cluster. Engage as many processing cores as possible for this node. The amount of RAM defines how much data gets read from the node's memory. If you overtax the resources available to your Master Node, you restrict the ability of your cluster to grow.

Other considerations:

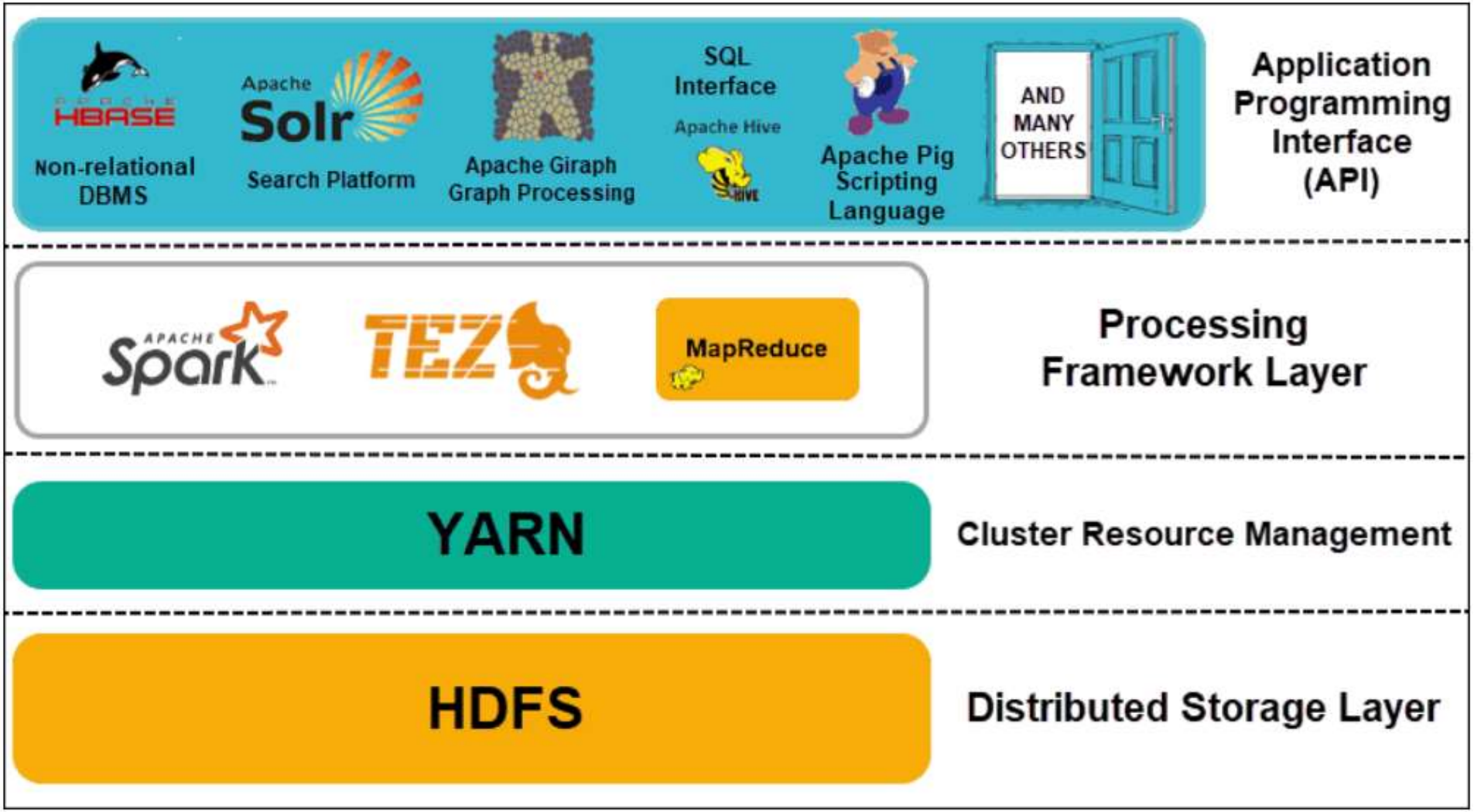
- Redundant power supplies: always reserved extra for the Master Node but not data nodes - they are expendable. For data nodes, use affordable dedicated servers that consume less power and produce less heat.
- Expect growth: addition of new nodes or disk space requires additional power, networking, and cooling.

Scaling Hadoop (Software)

New Hadoop-projects are being developed constantly, while existing ones are improved with more advanced features. Always keep an eye out for new developments on this front. Other considerations:

- The variety and volume of incoming datasets might mandate the introduction of additional frameworks.
- Implement new user-friendly tools that can solve technical problems (it is usually faster than trying to create a custom solution).
- Do not shy away from already developed commercial quick fixes.
- The market is saturated with vendors offering Hadoop-as-a-service or tailored standalone tools.

The Four Layers Hadoop Architecture



Ref: https://phoenixnap.com/kb/apache-hadoop-architecture-explaine

Hadoop Programming Models

- Distributed programming codes as well as the data at the computing nodes
- Use non-relational data model (avoid SQL Join)
- Programming models include:
  - MapReduce (also a core Processing Framework)
  - Spark (includes own Scheduler and Processing Frameworks)
  - Hbase
  - Hive
  - Pig

Apache Pig

Apache Pig is used to analyze large, unstructured, or semi-structured datasets. It allows users to write complex MapReduce transformations using a high-level SQL-like language called Pig Latin, which is easier to learn and requires significantly less code than Java MapReduce. Key notes:

- supports data transformation operations such as filtering, grouping, and joining.
- Converts Pig Latin scripts into executable MapReduce jobs (without Java programming)

Ref: https://www.geeksforgeeks.org/linux-unix/introduction-to-apache-pig/

Apache Hive

Apache Hive a Hadoop-based data warehouse infrastructure to structure, summarize, and analyze massive datasets stored in HDFS. Key notes:

- Converts HiveQL queries into MapReduce, Tez, or Spark jobs for processing.
- Uses a "metastore" to store schema information, enabling data abstraction.
- Supports partitioning and bucketing to optimize query speed.
- Primarily used for OLAP (Online Analytical Processing) rather than transactional (OLTP) workloads.
- Developed by Facebook to handle massive data, transferred to Apache.

Ref: https://www.databricks.com/blog/what-is-apache-hive

Apache Hbase

Apache HBase is a distributed, column-oriented NoSQL database that runs on top of the Hadoop Distributed File System (HDFS). It provides real-time, random read/write access to massive datasets (petabytes) and is designed for high-performance, fault-tolerant storage, often used for sparse data, Ideal for storing sparse data, real-time analytics, and high-volume data ingestion.

Ref: https://www.ibm.com/think/topics/hbase

Apache Spark

Apache Spark is considered a ***super-fast***, distributed data processing engine designed for large-scale data analytics, ***capable of processing data in-memory up to 100x faster than Hadoop MapReduce***. Unlike Hadoop, which writes to disk, Spark employs RAM for rapid, real-time analytics, machine learning, and stream processing. It is usually deployed with Hadoop (foundation), using HDFS for storage. It is ***more expensive to implement and scale***.

Ref: https://aws.amazon.com/compare/the-difference-between-hadoop-vs-spark

Next Class

- Spark Architecture
- Introduction to pySpark

Presentation Credits

- 2024, Kanlić, Fahir, Lecture Notes

- 2019, Ofer Mendelevitch, Casey Stella and Douglas Eadline, Practical Data Science with Hadoop and Spark: Designing and Building Effective Analytics at Scale, PowerPoint Slides